Estado y autoría

Los ADR distinguen decisiones confirmadas por el equipo, propuestas de diseño y partes pendientes. El equipo debe revisarlos para la defensa. La bitácora registra interacciones reales y revisiones técnicas internas; no inventa una revisión humana ni sesiones anteriores.

ADR-001: Domain Model como patrón principal

Campo	Contenido
Estado	Propuesta para revisión del equipo.
Contexto	Las cotizaciones conservan versiones y vigencia; el pedido cambia de estado; inventario y reservas deben mantenerse consistentes. Esas reglas reaparecen en varios casos de uso.
Alternativas	Transaction Script ofrece menor estructura inicial, con riesgo de duplicar reglas. Table Module organiza conjuntos de datos, pero coloca el ciclo de vida del pedido fuera de los objetos que representan el negocio. Domain Model encapsula reglas y conserva identidad y comportamiento.
Decisión	Proponer Domain Model en la escritura y consultas SQL con DTO propios en la lectura. No clasificar cualquier SELECT como Table Module ni introducir una combinación de patrones sin delimitarla.
Consecuencias	Beneficio: responsabilidades visibles y reglas reutilizables. Costo: más objetos, contratos y validación de límites. Riesgo: clases anémicas o servicios que absorban el dominio. Control: métodos de negocio y cinco hallazgos SOLID.
Trazabilidad	BN02/BN03; BR01–06; RNF02; US04/US05/US09/US10; CU02–CU05.

ADR002 Python y PostgreSQL alojado en Supabase

Campo	Contenido
Estado	Tecnología y organización confirmadas por el equipo; proyecto aún no creado.
Contexto	Se propuso PostgreSQL en Supabase y se autorizó continuar con esa opción; se eligió Santana’s Org como repositorio en Github para un proyecto nuevo.
Alternativas	SQLite simplifica una demostración local. PostgreSQL local conserva las funciones del motor pero requiere instalación/operación. Supabase aloja PostgreSQL y permite administración y persistencia compartida.
Decisión	Python organiza el dominio y la aplicación. Un adaptador PostgreSQL conserva transacciones y repositorios. Supabase se utiliza inicialmente para alojar la base; se conserva una única base de lectura y escritura.
Consecuencias	Beneficio: práctica con persistencia compartida y transacciones de PostgreSQL. Costo: conexión, permisos, dependencias y aprovisionamiento. El costo de creación debe verificarse antes de activar un proyecto; no se asume que una organización sea gratuita. No hay proyecto activo de FerreMax ni pruebas conectadas reportadas.
Trazabilidad	RNF02/03/04/08; BR04/05/16; CU04/CU05; conversación de decisión tecnológica y Release.

ADR003 Unit of Work para confirmación y reserva

Campo	Contenido
Estado	Propuesta implementada y revisada con Memory; validación PostgreSQL pendiente.
Contexto	AC-US09-01 exige reserva atómica y AC-US09-02 impide asignar la última unidad a dos pedidos. Guardar el estado del pedido separado de la reserva puede producir confirmaciones incoherentes.
Alternativas	Guardar por separado introduce cambios parciales. Coordinar con consistencia eventual añade recuperación y estados de compensación aún no definidos. Una transacción local puede proteger los cambios requeridos en una misma base.
Decisión	El servicio abre UoW, bloquea pedido e inventarios en orden, invoca al dominio, guarda pedido/reservas/auditoría/outbox y confirma una sola transacción. Ante error, revierte la operación completa.
Consecuencias	Beneficio: confirmación y compromiso de inventario quedan juntos. Costo: retención breve de bloqueos y manejo de contención. La transacción abarca más de un agregado sin convertirlos en una sola raíz. Las conexiones y los bloqueos reales necesitan ensayo PostgreSQL; RNF01 no se declara cumplido sin medirlo.

Campo	Contenido
Trazabilidad	BN02; BR04-06/BR16; RNF02/RNF04; AC-US06-01/02; AC-US09-01/02; CU04/CU05.

ADR004 CQRS básico sobre una sola base

Campo	Contenido
Estado	Propuesta implementada en comandos y consultas seleccionados.
Contexto	La consulta global debe distinguir física, comprometida, tránsito y disponible. Confirmar y cancelar cambian estado y exigen validaciones diferentes a una consulta.
Alternativas	Un modelo único para pantallas y escritura simplifica estructuras, pero puede permitir modificaciones involuntarias. Bases separadas exigirían sincronización y operación adicional. Separar handlers y modelos conservando PostgreSQL resuelve la necesidad actual.
Decisión	ConfirmarPedido y CancelarPedido son Commands. ConsultarDisponibilidad y ConsultarPedido son Queries con contratos de lectura y DTO propios. Los commands delegan a la Service Layer; una consulta no recibe métodos para guardar.
Consecuencias	Beneficio: intención explícita y lectores pequeños. Costo: DTO y handlers adicionales. No implica Event Sourcing, réplicas ni dos bases. La separación se aplica donde aporta claridad y no obliga a crear un handler decorativo por cada método.
Trazabilidad	BN01/BN02; BR04-06; RNF01/RNF06; US01/US02/US09/US10; CU01/CU04/CU05.

ADR005 Auditoría y outbox junto con el cambio de negocio

Campo	Contenido
Estado	Propuesta implementada como registro; consumidor externo pendiente.
Contexto	BR16 y RNF04 exigen quién, cuándo, qué cambió y por qué. Publicar un evento antes del COMMIT puede comunicar una confirmación que posteriormente falla.
Alternativas	Auditar/publicar por separado puede perder registros o emitir hechos no confirmados. Añadir un intermediario de mensajes externo aumenta trabajo sin un consumidor definido. Guardar auditoría y evento junto con el cambio permite conservar evidencia coherente.
Decisión	Registrar auditoría y evento en la misma transacción. El backend operativo tiene permiso de insertar y leer auditoría, sin actualizarla ni borrarla. Una outbox técnica conserva eventos pendientes para un consumidor posterior.
Consecuencias	Beneficio: coherencia entre cambio y evidencia. Costo: tabla adicional y necesidad futura de publicación/idempotencia. El prototipo registra eventos, pero no afirma entregar notificaciones ni garantizar exactamente una entrega externa.
Trazabilidad	BN10; BR16; RNF04; CA-US20-01; CU02-CU05.

Matriz de trazabilidad por necesidad

Necesidad	Reglas y RNF	Historias	Caso o alcance	Componentes	Decisiones	Cobertura
BN01	BR04; RNF01	US01/US02	CU01	ConsultarDisponibilidadHandler; DisponibilidadReader	ADR004	Consulta implementada; objetivo temporal sin medir.
BN02	BR04–06; RNF02	US09/US10	CU04/CU05	Pedido; InventarioUbicacion; UnitOfWork	ADR001/003	Reserva y liberación de contado en una ubicación.
BN03	BR01–03; RNF08	US03–05	CU02/CU03	Cotizacion; CotizacionService; CotizacionRepository	ADR001	Revalidación y conversión implementadas; emisión inicial simulada.
BN04	BR07/08	US05/US11/US14/US15	CU04 más diseño general	Pedido; PoliticaSurtidoMultiorigen; Transferencia	ADR001/003	Multiorigen y surtido diseñados; prototipo local.

Necesidad	Reglas y RNF	Historias	Caso o alcance	Componentes	Decisiones	Cobertura
BN05	BR09–11	US07/US08	Diseño comercial	CuentaCredito; PoliticaCredito	ADR001	Evaluación normal probada; excepciones y persistencia pendientes.
BN06	BR12/13	US11–13	Diseño logístico	Transferencia; InventarioUbicacion	ADR001/003	Mapeado; sin flujo ejecutable de transferencia.
BN07	BR08	US14	Diseño comercial	Pedido; PartidaPedido; aceptación del cliente	ADR001	Mapeado; sin aplicación ejecutable de parciales.
BN08	BR14; RNF07	US16/US17	Diseño logístico	Entrega; DireccionEntrega; EvidenciaEntrega	ADR001	Mapeado; sin programación de unidad ni captura operativa.
BN09	BR15; RNF08	US18	Diseño de postventa	Devolucion; PoliticaDevolucion; Categoria	ADR001	Política exacta pendiente del negocio.
BN10	BR16; RNF04	US19/US20	CU02–CU05 y diseño de panel	Auditoria; Outbox; DTO de reportes	ADR005	Auditoría local implementada; panel completo pendiente.

Matriz de trazabilidad por regla

Regla	Escenarios U2	Caso o alcance	Componente responsable	Evidencia o límite
BR01	US03/US04/US05	CU02/CU03	CotizacionService	No llama reservar durante revalidación/conversión; prueba sin stock comprometido.
BR02	US03/US04	CU02/CU03	Cotizacion; CotizacionRepository	Folio, versión y vigencia conservados; estado vigente se deriva de Vigencia.
BR03	AC-US04-01/02	CU02/CU03	Cotizacion.convertir/revalidar	Vencida rechazada; versión anterior conservada.
BR04	US02; AC-US09-01	CU01/CU04	InventarioUbicacion.disponible	Física menos reservas; tránsito informado por separado.
BR05	US06; AC-US09-01	CU04	Pedido.confirmar; InventarioUbicacion.reservar	Dominio exige confirmado; UoW guarda estado y reserva juntos.
BR06	AC-US10-01	CU05	Pedido.cancelar; InventarioUbicacion.liberar	Cancelación antes de surtido libera; reducción es variante futura.
BR07	US01/US11	Diseño general	PoliticaSurtidoMultiorigen	Prioridad local, almacén, otras sucursales; prototipo no implementa esta selección.
BR08	AC-US14-01/02	Diseño general	Pedido y PartidaPedido	Aceptación explícita para parciales; no usar excepción de stock como consentimiento.
BR09	AC-US07-01	Diseño general	CuentaCredito.evaluar	Activo es condición de autorización normal.
BR10	AC-US07-01/02	Diseño general	PoliticaCredito	Bloqueo o insuficiencia devuelven EnRevision.
BR11	AC-US08-01/02	Diseño general	CuentaCredito y decisión excepcional	Motivo, responsable, fecha y condición requeridos; no inventar umbrales de excepción.
BR12	AC-US11-01; US12	Diseño logístico	Transferencia	Origen, destino, partidas, responsable y estado.
BR13	AC-US13-01/02	Diseño logístico	Transferencia e InventarioUbicacion	Recepción actualiza tránsito/destino; diferencia se conserva como incidencia.
BR14	AC-US17-01/02	Diseño logístico	Entrega	Evidencia de receptor y resultado; incidencia no cierra como entregado.
BR15	CA-US18-01	Diseño postventa	Devolucion y PoliticaDevolucion	Evaluación por categoría/condición y política pendiente.

Regla	Escenarios U2	Caso o alcance	Componente responsable	Evidencia o límite
BR16	CA-US20-01	CU02–CU05	UnitOfWork.registrar; auditoria	Valores anteriores/nuevos, actor, fecha y motivo dentro de la transacción.

Trazabilidad de los rechazos

Escenario	Entrada o situación	Resultado y comprobación
AC-US04-01	Cotización fuera de vigencia	CotizacionVencida; no crea pedido.
AC-US06-02	Pago rechazado	Pedido sigue Pendiente; sin reserva ni evento de confirmación.
AC-US09-02	Dos pedidos intentan la última unidad	Uno confirma; otro recibe DisponibilidadInsuficiente. Memory probado; PostgreSQL pendiente.
AC-US10-01	Cancelación autorizada antes del surtido	Estado Cancelado y stock disponible restituido.
AC-US01-02 propuesto	SKU inexistente	encontrado falso y lista vacía; ID original repetido registrado en H01.

Bitácora de IA

Itera	Objetivo	Prompt	Respuesta útil	Error o supuesto detectado	Decisión del equipo
IA01	Decidir el patrón de lógica de negocio a partir de las reglas de negocio y los cinco casos de uso.	"Analiza la siguiente información. Se trata de reglas de negocio, y cinco casos de uso para el diseño arquitectural de un sistema. Compara Transaction Script, Table Module y Domain Model, indica las ventajas y desventajas y en base a eso y posibles consecuencias, indica cuál consideras el patrón ideal para utilizar."	Resumen de los tres patrones de lógica de negocio, detalles de ventajas, desventajas y consecuencias de cada patrón, y un análisis profundo del negocio en cada patrón. Al final, IA consideró Domain Model como patrón ideal.	Se encontraron OB01/CE01 incongruentes, además de pequeños errores en las descripciones de los patrones, pero en términos generales se desarrolló correctamente.	Se analizó el resumen de IA y se comparó con la idea de lo visto en clase, se corroboró la idea del equipo y se decidió por Domain Model como patrón a usar.
IA02	Construir el dominio del sistema.	"Según el patrón de lógica escogido, analiza este glosario técnico (glosario corregido de la unidad 3) y las reglas de negocio, propón candidatos a entidades, value objects, aggregates y domain events. Incluye la evidencia para cada candidato y las dudas o incongruencias que encuentres para validación humana"	Lista de varios candidatos para cada elemento del dominio con sus respectivas evidencias de reglas y necesidades, así como dudas que necesitan validación del equipo.	Algunos candidatos se escapaban de las reglas y el glosario de negocio, por lo que se descartaron. También hubo ideas que eran redundantes e ilógicas.	Se seleccionaron candidatos que no se escapaban de las reglas y el glosario del negocio y que concordaban con el flujo establecido en la iteración 2. Se seleccionaron solo las dudas que fueran realmente relevantes.
IA03	Construir la Service Layer	"Se eligieron los siguientes candidatos para los elementos del dominio (elementos elegidos de la interacción anterior). A partir de estos y de las reglas anteriores, propón Application Services."	Entregó una tabla con varios servicios, enlazados con sus reglas, entradas, salidas y colaboradores.	Algunos servicios no estaban respaldados por reglas y otros no tenían congruencia con las mismas.	Hubo servicios suficientes para elegirlos para la tabla de service layer, y se corroboraron los enlaces con sus elementos.
IA04	Encontrar incongruencias SOLID y validar la arquitectura diseñada.	"Analiza el siguiente diseño arquitectural (dominio, service layer y reglas de negocio, UoW, etc.) y busca violaciones a SOLID. No reescribas información ni modifiques lo adjunto, sólo señala las violaciones, identifica el riesgo con evidencia y propón alternativas para solucionarlas"	Encontró una cantidad de violaciones SOLID e identificó a cuáles elementos hacía referencia. Dio evidencia y propuso alternativas para solucionarlo.	Una violación era incongruente por lo que se descartó. El resto era correcta y solucionables de manera sencilla.	Se seleccionaron aquellas violaciones que requerían cambios mayores para incluirlas. Aquellas que no, se solucionaron fácilmente y se omitieron.

Itera	Objetivo	Prompt	Respuesta útil	Error o supuesto detectado	Decisión del equipo
IA05	Diseñar diagramas con IA para revisarlos y pasarlos a PlantUML.	"En base al dominio del negocio (Se adjunta información del dominio) construye un diagrama de clases. No agregues ninguna clase adicional ni ningún elemento que no esté en esta información, y mantén la cardinalidad tal como está."	Un diseño inicial de diagrama de clase del dominio utilizado en el negocio.	Agregó una relación adicional entre dos clases no relacionadas. El resto fue un diagrama construido correctamente.	Se indicó a la IA que eliminara la relación sobrante y se construyó otro diagrama correctamente.
IA06	Realizar diagramas de secuencia iniciales para el flujo completo y flujo interrumpido.	"A partir del siguiente BPMN del negocio, [BPMN de la unidad 2] y las reglas del dominio construido, además del dominio construido, construye un diagrama de secuencia de un proceso completo y un proceso interrumpido por falta de disponibilidad. No incluyas actores ni procesos no incluidos en el dominio"	Un diagrama de secuencia de un flujo completo de inicio a fin, y diagrama de secuencia interrumpido por falta de disponibilidad.	Los diagramas eran simples pero correctos, sólo que no indicaban explícitamente los actores.	Se indicó a la IA agregar los actores sin modificar lo demás, y se entregaron diagramas correctos.
IA07	Cambiar los diagramas de IA a código para PlantUML.	"Utiliza este diagrama (diagrama realizado y corregido por IA) y conviértelo a código para realizarlo en PlantUML. No cambies ninguna información que aparezca en el diagrama" (Se usó este prompt para los diagramas realizados por IA)	Código para realizar los mismos diagramas en PlantUML.	El código era correcto para realizarlo en PlantUML.	Se utilizó el código en PlantUML y se verificó que no se perdiera información.